

Шаг 1: Создание сущности Abiturient и подсчет баллов

Красная фаза (Red):

Был написан тест `testAbiturientTotalScore()`, который создает объект `Abiturient`, добавляет ему оценки (10, 8, 9) и проверяет, что сумма баллов равна 27. Код не компилировался, так как класса `Abiturient` еще не существовало.

Зеленая фаза (Green):

Создан класс `Abiturient` с полями `name` и `grades`. Реализован метод `getTotalScore()`, который суммирует элементы списка. Тест успешно пройден.

Шаг 2: Обработка исключительных ситуаций

Красная фаза (Red):

Написан тест `testNegativePlacesThrowException()`. Он вызывает метод отбора абитуриентов `getAdmittedList`, передавая отрицательное количество мест. Ожидается выброс исключения `IllegalArgumentException`.

Зеленая фаза (Green):

Создан класс `AdmissionSystem` с методом `getAdmittedList`. В начало метода добавлена проверка: `if (places < 0) throw new IllegalArgumentException(...)`. Тест стал зеленым.

Шаг 3: Основная логика отбора (мест меньше, чем абитуриентов)

Красная фаза (Red):

Написан тест `testAdmittedListWhenPlacesLessThanApplicants()`. Создано 3 абитуриента с суммами баллов: 30, 15, 26. Количество мест — 2. Тест ожидает, что метод вернет список из 2-х человек в порядке убывания баллов. Тест упал, так как метод возвращал `null`.

Зеленая фаза (Green):

В методе `getAdmittedList` реализована логика: список абитуриентов сортируется по убыванию баллов с помощью `Comparator`, после чего возвращается подсписок размером `places`. Тест пройден.

Шаг 4: Пограничный случай (мест больше или равно числу абитуриентов)

Красная фаза (Red):

Добавлен тест `testAdmittedListWhenPlacesMoreThanApplicants()`. 2 абитуриента, но мест 5. Ожидается, что вернутся оба абитуриента без ошибки выхода за пределы массива (`IndexOutOfBoundsException`). Тест упал.

Зеленая фаза (Green):

В метод `getAdmittedList` добавлена математическая функция `Math.min(places, applicants.size())`, которая ограничивает длину возвращаемого списка. Тест стал зеленым.